Alpaca Options Lab: Comprehensive Audit & Action Plan

Executive Summary

This document provides a comprehensive audit of the `alpaca-options-lab` component based on the technical requirements outlined in the ARM64 HFT roadmap, along with a detailed implementation plan. We focus on production-grade options trading infrastructure excluding Oracle Cloud-specific optimizations.

Part 1: Comprehensive Audit

1.1 Critical Components Assessment

A. Data Architecture & Feed Handling

Expected Requirements:

- Real-time options chain data ingestion
- Option symbology normalization (OSI standard)
- Efficient storage of Greeks, bid/ask, volume
- Historical data retrieval for backtesting

Potential Gaps:

1. **Symbol Normalization** - OSI 21-character strings need integer mapping
    - Problem: Storing raw strings wastes storage and slows queries
    - Impact: Database bloat, slower backtesting
2. **Feed Handler Efficiency** - Python GIL bottleneck for high-frequency data
    - Problem: Single-threaded processing limits throughput
    - Impact: Missing ticks during volatility spikes
3. **Data Validation** - No mention of stale data detection
    - Problem: Using outdated quotes in fast markets
    - Impact: Incorrect Greeks calculations, bad fills

Recommendations:

```
python
```

```python
# Symbol normalization layer
class SymbolMapper:
    def __init__(self):
        self._symbol_to_id = {}
        self._id_to_symbol = {}

    def register(self, osi_symbol: str) -> int:
        """Convert SPY251220C00400000 -> integer ID"""
        if osi_symbol not in self._symbol_to_id:
            id = len(self._symbol_to_id) + 1
            self._symbol_to_id[osi_symbol] = id
            self._id_to_symbol[id] = osi_symbol
        return self._symbol_to_id[osi_symbol]
```

B. Greeks Calculation Engine

Expected Requirements:

- Real-time IV solving (Newton-Raphson or Brent's method)
- Greeks: Delta, Gamma, Theta, Vega, Rho
- Lazy evaluation to avoid redundant calculations
- Support for American options (early exercise premium)

Potential Gaps:

1. **Performance Bottleneck** - Pure Python IV solver too slow
    - Problem: 1000+ options per second requires <1ms per calculation
    - Impact: Stale Greeks, delayed risk decisions
2. **Accuracy Issues** - Naive Black-Scholes doesn't handle dividends
    - Problem: Wrong pricing for high-dividend stocks
    - Impact: Mispriced positions, incorrect hedging
3. **No Volatility Surface** - Flat IV assumption breaks for skew
    - Problem: Real markets have volatility smile/skew
    - Impact: Systematic mispricing of OTM options

Recommendations:

```python
# C++ accelerated pricer with PyBind11
# File: greeks_engine.cpp
#include <pybind11/pybind11.h>
#include <cmath>

double black_scholes_call(double S, double K, double T,
                          double r, double sigma) {
    double d1 = (log(S/K) + (r + 0.5*sigma*sigma)*T) / (sigma*sqrt(T));
    double d2 = d1 - sigma*sqrt(T);
    return S*norm_cdf(d1) - K*exp(-r*T)*norm_cdf(d2);
}

PYBIND11_MODULE(greeks_engine, m) {
    m.def("bs_call", &black_scholes_call);
}
```

C. Option Lifecycle Manager (OLM)

Expected Requirements:

- Finite State Machine for position lifecycle
- Expiration/assignment logic
- Rolling strategies automation
- Risk limit monitoring (position-level and portfolio-level)

Potential Gaps:

1. **No FSM Implementation** - Ad-hoc state tracking
    - Problem: Invalid state transitions (e.g., rolling closed positions)
    - Impact: Logic errors, orphaned positions
2. **Missing Assignment Risk Detection** - No ITM monitoring near expiration
    - Problem: Surprise assignments drain capital
    - Impact: Margin calls, forced liquidations
3. **No Atomic Rolling** - Two-leg rolls can partially fail
    - Problem: Close leg succeeds, open leg fails

- Impact: Unhedged exposure

Recommendations:

```python
# FSM-based position manager
from enum import Enum
from dataclasses import dataclass
from typing import Optional

class PositionState(Enum):
    PENDING = "pending"
    ACTIVE = "active"
    ROLLING = "rolling"
    ASSIGNMENT_RISK = "assignment_risk"
    EXPIRED = "expired"
    CLOSED = "closed"

@dataclass
class OptionPosition:
    contract_id: int
    quantity: int
    state: PositionState
    entry_price: float
    current_greeks: Optional[dict] = None

    def can_transition(self, new_state: PositionState) -> bool:
        """Validate state transitions"""
        transitions = {
            PositionState.PENDING: [PositionState.ACTIVE, PositionState.CLOSED],
            PositionState.ACTIVE: [PositionState.ROLLING, PositionState.ASSIGNMENT_RISK, PositionState.CLOSED],
            PositionState.ROLLING: [PositionState.ACTIVE, PositionState.CLOSED],
            PositionState.ASSIGNMENT_RISK: [PositionState.EXPIRED, PositionState.CLOSED],
        }
        return new_state in transitions.get(self.state, [])
```

D. Backtesting Framework

Expected Requirements:

- Event-driven architecture (no look-ahead bias)

- Realistic execution simulation (slippage, partial fills)
- Option-specific phenomena (assignment, pin risk, 0DTE spreads)
- Performance metrics (Sharpe, max drawdown, Greeks exposure over time)

Potential Gaps:

1. **Vectorized Backtesting** - If using pandas.apply()
    - Problem: Cannot model path dependency of American options
    - Impact: Overly optimistic backtest results
2. **No Spread Modeling** - Using midpoint execution
    - Problem: Illiquid options have wide spreads
    - Impact: 10-20% overestimation of returns
3. **Missing Pin Risk Logic** - No special handling for 0DTE at strikes
    - Problem: Real traders face liquidity crunches near expiration
    - Impact: Backtest shows profit, live trading fails

Recommendations:

```python
```

```python
# Event-driven backtester core
from collections import deque
from dataclasses import dataclass
from datetime import datetime

@dataclass
class MarketEvent:
    timestamp: datetime
    contract_id: int
    bid: float
    ask: float
    last: float

class EventDrivenBacktester:
    def __init__(self):
        self.event_queue = deque()
        self.current_time = None
        self.portfolio = {}

    def process_event(self, event: MarketEvent):
        """Strict causality - no future peeking"""
        self.current_time = event.timestamp

        # Check for expirations BEFORE processing ticks
        self._check_expirations()

        # Update Greeks only if price moved > threshold
        if self._should_recalc_greeks(event):
            self._update_greeks(event)

        # Generate signals
        signals = self.strategy.on_tick(event, self.portfolio)

        for signal in signals:
            self._execute_with_slippage(signal)
```

E. Risk Management

Expected Requirements:

- Portfolio Greeks aggregation (net delta, gamma, theta, vega)

- Margin requirement calculation
- Position sizing based on Kelly criterion or volatility targeting
- Circuit breakers for max daily loss, max position delta

Potential Gaps:

1. **No Aggregated Greeks** - Only per-position tracking
    - Problem: Portfolio could be short 1000 deltas without knowing
    - Impact: Catastrophic losses on large moves
2. **Missing Margin Model** - Static allocation instead of dynamic
    - Problem: Broker margin requirements change with volatility
    - Impact: Surprise liquidations
3. **No Portfolio Optimization** - Equal weighting or manual allocation
    - Problem: Suboptimal risk-adjusted returns
    - Impact: Over-concentration in correlated positions

Recommendations:

python

```python
# Portfolio risk aggregator
import numpy as np
from scipy.optimize import minimize

class PortfolioRiskManager:
    def __init__(self, max_delta=100, max_loss_pct=0.02):
        self.max_delta = max_delta
        self.max_loss_pct = max_loss_pct

    def aggregate_greeks(self, positions: list) -> dict:
        """Sum Greeks across all positions"""
        return {
            'delta': sum(p.quantity * p.greeks['delta'] for p in positions),
            'gamma': sum(p.quantity * p.greeks['gamma'] for p in positions),
            'theta': sum(p.quantity * p.greeks['theta'] for p in positions),
            'vega': sum(p.quantity * p.greeks['vega'] for p in positions)
        }

    def check_limits(self, portfolio_greeks: dict) -> tuple[bool, str]:
        """Circuit breaker logic"""
        if abs(portfolio_greeks['delta']) > self.max_delta:
            return False, f"Delta limit breached: {portfolio_greeks['delta']}"
        return True, "OK"

    def optimize_allocation(self, expected_returns, cov_matrix):
        """Mean-variance optimization with constraints"""
        n = len(expected_returns)

        def objective(weights):
            return -weights @ expected_returns  # Negative for maximization

        constraints = [
            {'type': 'eq', 'fun': lambda w: np.sum(w) - 1},  # Fully invested
            {'type': 'ineq', 'fun': lambda w: w}  # No shorts
        ]

        result = minimize(objective, np.ones(n)/n, constraints=constraints)
        return result.x
```

F. Storage & Database

Expected Requirements:

- Time-series optimized (TimescaleDB or similar)
- Compressed historical data (>90% compression ratio)
- Fast tick ingestion (50k+ ticks/sec)
- Efficient analytical queries (OHLC aggregation, volatility surfaces)

Potential Gaps:

1. **Using Standard PostgreSQL** - No time-series optimizations
    - Problem: Slow queries on large date ranges
    - Impact: 10-60 second backtests instead of milliseconds
2. **No Compression** - Storing raw ticks
    - Problem: 1 year SPY options = 500GB+ uncompressed
    - Impact: Storage costs, slow queries
3. **Write Bottleneck** - Single-threaded inserts
    - Problem: Cannot keep up with OPRA feed
    - Impact: Missing data, gaps in backtests

Recommendations:

sql

```sql
-- TimescaleDB schema
CREATE TABLE option_contracts (
    id SERIAL PRIMARY KEY,
    osi_symbol TEXT UNIQUE,
    root_symbol TEXT,
    expiration DATE,
    strike NUMERIC,
    option_type CHAR(1)
);

CREATE TABLE option_ticks (
    time TIMESTAMPTZ NOT NULL,
    contract_id INT REFERENCES option_contracts(id),
    bid NUMERIC,
    ask NUMERIC,
    last NUMERIC,
    volume INT,
    iv NUMERIC
);

-- Convert to hypertable
SELECT create_hypertable('option_ticks', 'time',
    chunk_time_interval => INTERVAL '1 day');

-- Enable compression
ALTER TABLE option_ticks SET (
    timescaledb.compress,
    timescaledb.compress_segmentby = 'contract_id',
    timescaledb.compress_orderby = 'time'
);

-- Auto-compress old data
SELECT add_compression_policy('option_ticks',
    INTERVAL '7 days');
```

G. Visualization & Monitoring

Expected Requirements:

- Real-time Greeks charts (portfolio and per-position)
- Options chain heatmap (volume, OI, IV)

- PnL waterfall over time
- Risk dashboard (current exposure vs limits)

Potential Gaps:

1. **Static Charts** - Using Matplotlib/Seaborn
    - Problem: Cannot interact with data (zoom, pan)
    - Impact: Poor UX, slow analysis
2. **No Real-Time Updates** - Manual refresh
    - Problem: Stale data in fast markets
    - Impact: Delayed risk decisions
3. **Missing Context** - Charts without overlays (support/resistance, events)
    - Problem: Hard to correlate PnL with market events
    - Impact: Slower hypothesis generation

Recommendations:

python

```python
# Lightweight Charts integration
from lightweight_charts import Chart

class OptionsDashboard:
    def __init__(self):
        self.chart = Chart(width=1200, height=600)
        self.greeks_pane = self.chart.create_subchart(height=200)

    def update_pnl(self, timestamp, pnl):
        """Real-time PnL updates"""
        self.chart.update([{'time': timestamp, 'value': pnl}])

    def plot_greeks(self, greeks_history):
        """Multi-line Greeks chart"""
        for greek_name, values in greeks_history.items():
            self.greeks_pane.line(values, name=greek_name)

    def add_event_marker(self, timestamp, label):
        """Mark expirations, earnings, etc."""
        self.chart.marker(timestamp, text=label,
                          shape='circle', color='red')
```

1.2 Code Quality Assessment

Expected Standards:

- Type hints on all functions
- Docstrings with examples
- Unit tests (>80% coverage)
- Integration tests for critical paths
- Logging at appropriate levels
- Error handling with specific exceptions

Potential Issues:

1. **No Type Safety** - Using bare `dict` instead of `TypedDict` or `dataclasses`
2. **Missing Tests** - No tests for Greeks accuracy or FSM transitions
3. **Poor Error Messages** - Generic exceptions without context

4. **No Observability** - Print statements instead of structured logging

Recommendations:

```python
```

```python
# Type-safe data models
from dataclasses import dataclass
from datetime import date
from typing import Literal

@dataclass
class OptionContract:
    symbol: str
    underlying: str
    expiration: date
    strike: float
    option_type: Literal['C', 'P']

    def to_osi(self) -> str:
        """Convert to 21-character OSI format"""
        exp_str = self.expiration.strftime('%y%m%d')
        strike_str = f"{int(self.strike * 1000):08d}"
        return f"{self.underlying:6} {exp_str} {self.option_type} {strike_str}"

# Structured logging
import structlog

logger = structlog.get_logger()

def calculate_iv(market_price, S, K, T, r):
    try:
        iv = newton_iv_solver(market_price, S, K, T, r)
        logger.info("iv_calculated",
                    market_price=market_price,
                    iv=iv,
                    strike=K)
        return iv
    except ValueError as e:
        logger.error("iv_calculation_failed",
                     error=str(e),
                     market_price=market_price,
                     strike=K)
        raise
```

Part 2: Action Plan

Phase 1: Foundation (Weeks 1-2)

Goal: Establish core data infrastructure and symbol management

1.1 Database Setup

- [ ] Install TimescaleDB (Docker or local)
- [ ] Create schema with hypertables
- [ ] Implement batch insert writer (asyncpg)
- [ ] Set up compression policies

Deliverables:

- `schema.sql` - Complete database schema
- `db_manager.py` - Async database client
- Write performance benchmark: 50k ticks/sec

1.2 Symbol Normalization

- [ ] Build `SymbolMapper` class
- [ ] Implement OSI parser (regex or struct-based)
- [ ] Create contract definition cache
- [ ] Add vendor-specific handlers (Alpaca, Databento)

Deliverables:

- `symbology.py` - Symbol normalization module
- Unit tests with 100+ OSI samples
- Performance: <10μs per parse

1.3 Feed Handler Foundation

- [ ] Implement Alpaca WebSocket client
- [ ] Add tick validation (staleness, sanity checks)
- [ ] Create in-memory ring buffer (for future C++ integration)
- [ ] Build market data cache (last N ticks)

Deliverables:

- `feed_handler.py` - Async feed client

- Integration test with paper trading account
- Latency measurement: P50, P99, P999

Phase 2: Greeks Engine (Weeks 3-4)

Goal: Accurate, fast option pricing and Greeks

2.1 Pure Python Implementation

- [ ] Black-Scholes pricer with dividends
- [ ] Newton-Raphson IV solver
- [ ] Analytical Greeks formulas
- [ ] Unit tests against QuantLib

Deliverables:

- `pricing.py` - Pure Python pricer
- Accuracy tests (max error < 0.01%)
- Benchmark: ~100 options/sec

2.2 C++ Acceleration (Optional but Recommended)

- [ ] Write C++ pricing module
- [ ] PyBind11 bindings
- [ ] SIMD optimization (SSE/AVX for x86, NEON for ARM)
- [ ] Performance comparison

Deliverables:

- `greeks_engine.cpp` - Optimized pricer
- Python bindings in `pricing_ext.py`
- Benchmark: 10,000+ options/sec

2.3 Lazy Evaluation

- [ ] Build Greeks cache with TTL
- [ ] Implement delta/gamma approximation
- [ ] Add recalculation triggers (price move, time decay)

Deliverables:

- `lazy_greeks.py` - Caching layer
- Performance test: 90% cache hit rate

Phase 3: Option Lifecycle Manager (Weeks 5-6)

Goal: Robust position tracking with FSM

3.1 Core FSM

- [ ] Define `PositionState` enum
- [ ] Implement `OptionPosition` dataclass
- [ ] Build state transition validator
- [ ] Add state persistence (database)

Deliverables:

- `olm/fsm.py` - State machine
- FSM diagram (Mermaid/GraphViz)
- Unit tests for all transitions

3.2 Assignment Logic

- [ ] ITM detection at expiration
- [ ] Early assignment probability model
- [ ] Auto-exercise for long positions
- [ ] Margin requirement calculator

Deliverables:

- `olm/assignment.py` - Assignment handler
- Test cases with edge scenarios (pin risk, after-hours moves)

3.3 Rolling Automation

- [ ] Atomic two-leg order placement
- [ ] Rollback on partial fill
- [ ] Rolling criteria (time-based, delta-based)
- [ ] Roll suggestion engine

Deliverables:

- `olm/rolling.py` - Roll manager
- Integration test with paper trading

Phase 4: Risk Management (Weeks 7-8)

Goal: Portfolio-level risk monitoring and optimization

4.1 Greeks Aggregation

- [ ] Real-time portfolio Greeks calculator
- [ ] Greeks by expiration/strike breakdown
- [ ] Delta-equivalent position sizing

Deliverables:

- `risk/aggregator.py` - Portfolio Greeks
- Real-time dashboard widget

4.2 Risk Limits

- [ ] Position-level limits (max delta per contract)
- [ ] Portfolio-level limits (net delta, gamma)
- [ ] Capital allocation rules (max % per strategy)
- [ ] Circuit breakers (max daily loss)

Deliverables:

- `risk/limits.py` - Limit checker
- Alert system (email/SMS/Slack)

4.3 Portfolio Optimization

- [ ] Integrate Riskfolio-Lib
- [ ] Hierarchical Risk Parity implementation
- [ ] Rebalancing scheduler
- [ ] Backtest optimization strategy

Deliverables:

- `risk/optimizer.py` - HRP allocator
- Comparison report (equal-weight vs HRP)

Phase 5: Backtesting Engine (Weeks 9-11)

Goal: Event-driven backtest with option-specific logic

5.1 Core Event Loop

- [ ] Event queue with priority
- [ ] Market/Signal/Order/Fill event classes
- [ ] DataHandler interface for tick replay
- [ ] Strategy base class

Deliverables:

- `backtesting/engine.py` - Core loop
- Example strategy (simple covered call)

5.2 Execution Simulation

- [ ] Slippage models (constant, volume-based)
- [ ] Partial fill logic
- [ ] Commission calculator
- [ ] Market hours constraints

Deliverables:

- `backtesting/execution.py` - Fill simulator
- Calibration against real fills

5.3 Option-Specific Logic

- [ ] Expiration/assignment handler
- [ ] 0DTE spread widening
- [ ] Pin risk simulator
- [ ] Dividend adjustments

Deliverables:

- `backtesting/options.py` - Options logic

- Validation against historical data

5.4 Performance Metrics

- [ ] Sharpe/Sortino ratio
- [ ] Max drawdown (MDD)
- [ ] Win rate, profit factor
- [ ] Greeks exposure over time

Deliverables:

- `backtesting/metrics.py` - Analytics
- HTML report generator

Phase 6: Visualization (Weeks 12-13)

Goal: Interactive, real-time dashboards

6.1 Main Trading Dashboard

- [ ] Real-time PnL chart
- [ ] Options chain heatmap
- [ ] Positions table with live Greeks
- [ ] Order book depth (if available)

Deliverables:

- `ui/dashboard.py` - Main UI
- WebSocket updates (Socket.IO or similar)

6.2 Greeks Visualization

- [ ] Multi-line portfolio Greeks chart
- [ ] Greeks by expiration stacked area chart
- [ ] Delta/Gamma surface plots

Deliverables:

- `ui/greeks_charts.py` - Greeks UI
- Integration with Lightweight Charts

6.3 Backtest Analysis

- [ ] Equity curve with drawdowns
- [ ] Greeks exposure over time
- [ ] Trade-by-trade breakdown
- [ ] Comparison tool (multiple strategies)

Deliverables:

- `ui/backtest_viewer.py` - Analysis UI
- Export to PDF/HTML

Phase 7: Integration & Testing (Weeks 14-15)

Goal: End-to-end validation and hardening

7.1 Integration Tests

- [ ] Full cycle test (data ingestion → signal → order → fill → PnL)
- [ ] Multi-day backtest with real data
- [ ] Stress test (high volatility scenarios)
- [ ] Failure mode testing (API outage, database crash)

7.2 Performance Optimization

- [ ] Profile CPU/memory bottlenecks
- [ ] Optimize database queries
- [ ] Add caching layers
- [ ] Parallel processing where applicable

7.3 Documentation

- [ ] Architecture diagram
- [ ] API documentation (Sphinx)
- [ ] User guide for each module
- [ ] Example notebooks

Deliverables:

- Test report with coverage metrics
- Performance benchmark document

- Complete documentation site

Phase 8: Production Readiness (Week 16)

Goal: Deploy to paper trading, then live

8.1 Observability

- [ ] Prometheus metrics export
- [ ] Grafana dashboards
- [ ] Error tracking (Sentry or similar)
- [ ] Log aggregation (ELK or Loki)

8.2 Paper Trading

- [ ] Deploy to staging environment
- [ ] Run 1 week of paper trading
- [ ] Compare paper vs backtest results
- [ ] Fix discrepancies

8.3 Live Trading Preparation

- [ ] Add kill switch (emergency shutdown)
- [ ] Set conservative limits
- [ ] Create runbook for common issues
- [ ] Backup and disaster recovery plan

Deliverables:

- Production deployment checklist
- Monitoring runbook
- Post-mortem template

Part 3: Technology Stack

Core Dependencies

```toml
```

```toml
# pyproject.toml
[tool.poetry.dependencies]
python = "^3.11"

# Data & Numerical
numpy = "^1.24"
pandas = "^2.0"
scipy = "^1.10"

# Options Pricing
py_vollib = "^1.0"  # IV & Greeks reference

# Async & Networking
asyncio = "^3.4"
asyncpg = "^0.28"  # Fast Postgres driver
aiohttp = "^3.8"
websockets = "^11.0"

# Database
psycopg2-binary = "^2.9"
sqlalchemy = "^2.0"

# ML & Portfolio
scikit-learn = "^1.3"
riskfolio-lib = "^5.0"

# Visualization
lightweight-charts = "^1.0"
plotly = "^5.14"

# Trading APIs
alpaca-py = "^0.11"

# Utilities
structlog = "^23.1"
pydantic = "^2.0"
click = "^8.1"  # CLI

[tool.poetry.dev-dependencies]
pytest = "^7.3"
pytest-asyncio = "^0.21"
pytest-cov = "^4.1"
black = "^23.3"
```

```
ruff = "^0.0.270"
mypy = "^1.3"
```

Infrastructure

yaml

```yaml
# docker-compose.yml
version: '3.8'

services:
  timescaledb:
    image: timescale/timescaledb:latest-pg15
    ports:
      - "5432:5432"
    environment:
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - timescale_data:/var/lib/postgresql/data
      - ./schema.sql:/docker-entrypoint-initdb.d/schema.sql
    shm_size: 1g
    command: postgres -c shared_buffers=2GB -c max_connections=200

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data

  prometheus:
    image: prom/prometheus:latest
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
      - prometheus_data:/prometheus

  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
    environment:
      GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_PASSWORD}
    volumes:
      - grafana_data:/var/lib/grafana

volumes:
  timescale_data:
  redis_data:
```

```
  prometheus_data:
  grafana_data:
```

Part 4: Success Metrics

Technical Metrics

Metric	Target	Measurement
Tick Ingestion Rate	>50k/sec	Benchmark script
Greeks Calc Latency	<1ms P99	perf profiling
Database Write Latency	<100µs P99	asyncpg logging
Backtest Speed	1 year in <5 min	Timer
Data Compression	>85%	pg_total_relation_size
Test Coverage	>85%	pytest-cov
Type Coverage	>90%	mypy --strict

Trading Metrics

Metric	Target	Measurement
Sharpe Ratio	>1.5	Backtest
Max Drawdown	<15%	Backtest
Win Rate	>60%	Backtest
Profit Factor	>1.8	Backtest
Greeks Accuracy	<1% error	QuantLib comparison
Paper vs Backtest Correlation	>0.95	Live paper trading

Part 5: Risk Considerations

Technical Risks

1. **Data Quality** - Alpaca options data may have gaps
    - Mitigation: Cross-validate with Polygon or Databento
2. **API Rate Limits** - Alpaca has request limits
    - Mitigation: Implement exponential backoff, cache aggressively
3. **Latency Variability** - Cloud deployments have jitter
    - Mitigation: Use dedicated instances, monitor P99 latencies

Trading Risks

1. **Pin Risk** - 0DTE positions near strikes
    - Mitigation: Auto-close 1 hour before expiration
2. **Liquidity Risk** - Wide spreads on illiquid options
    - Mitigation: Only trade options with volume >100, OI >500
3. **Model Risk** - Black-Scholes assumptions violated
    - Mitigation: Use realized vol surfaces, stress test Greeks

---

Part 6: Next Steps

Immediate Actions (This Week)

1. **Repository Structure**

```
alpaca-options-lab/
├── src/
│   ├── data/
│   │   ├── feed_handler.py
│   │   ├── symbology.py
│   │   └── db_manager.py
│   ├── pricing/
│   │   ├── black_scholes.py
│   │   ├── greeks.py
│   │   └── iv_solver.py
│   ├── olm/
│   │   ├── fsm.py
│   │   ├── position.py
│   │   └── assignment.py
│   ├── risk/
│   │   ├── aggregator.py
│   │   ├── limits.py
│   │   └── optimizer.py
│   ├── backtesting/
│   │   ├── engine.py
│   │   ├── execution.py
│   │   └── metrics.py
│   └── ui/
│       ├── dashboard.py
│       └── charts.py
├── tests/
├── docs/
├── config/
├── scripts/
├── pyproject.toml
├── docker-compose.yml
└── README.md
```

2. **Create Initial Tests**

```bash
pytest tests/ --cov=src --cov-report=html
```

3. **Set Up CI/CD**
    - GitHub Actions for tests

- Pre-commit hooks (black, ruff, mypy)

Monthly Milestones

- **Month 1:** Core infrastructure + Greeks engine
- **Month 2:** OLM + Risk management
- **Month 3:** Backtesting engine
- **Month 4:** UI + Production readiness

Conclusion

This audit identified critical gaps in typical options trading implementations and provided a comprehensive 16-week action plan. Focus areas:

1. **Data Layer:** TimescaleDB with compression
2. **Pricing Engine:** C+